

Metrics that Matter

A Layered Measurement Strategy for Product, Platform, and Delivery Signals

1. Purpose and Principles

This strategy establishes a common measurement architecture across the software delivery organization. It defines three distinct but linked layers of signal: what users do (product analytics), how the product behaves (technical telemetry), and how the organization delivers (programmatic metrics). Revenue and conversion measures are deliberately out of scope; this framework measures experience, reliability, and flow.

Five principles govern collection and use of every metric in this document:

Metrics are system health indicators, not scorecards. No metric in this framework is used to rank individuals or teams. Programmatic metrics in particular are diagnostic instruments for improving the delivery system.

Segment before you aggregate. Aggregate numbers hide failure. Every technical metric is collected per endpoint, per service, and per client cohort, then rolled up. An aggregate 2% error rate can conceal one endpoint failing 40% of the time.

Trend deltas, not absolutes. Most signals carry a normal baseline. The actionable event is a step change, especially one correlated with a deployment, release, or policy change.

Alert on what pages people; trend the rest. Only SLO-bound reliability signals wake engineers. Everything else is reviewed on a defined cadence.

Baseline each converged organization separately. Legacy organizations enter with different tooling, pipelines, and debt. Org-wide targets are set only after per-organization baselines are established, so early targets do not punish teams that inherited the weakest pipelines.

2. The Three-Layer Architecture

Each layer answers a different question and has a different owner, cadence, and toolchain. The layers are collected independently but joined through shared dimensions (release version, service, endpoint, user cohort) so that a signal in one layer can be traced to its cause or consequence in another.

Layer	Question Answered	Primary Owner	Review Cadence
1. Product Analytics	What are users doing, and are they succeeding?	Product and UX	Weekly; cohort reviews monthly
2. Technical Telemetry	How is the product behaving in production?	Engineering and SRE	Real-time alerting; weekly review
3. Programmatic Delivery	How effectively does the organization deliver change?	Engineering leadership	Per sprint; quarterly baseline review

3. Layer 1: Product Analytics (What Users Do)

Behavioral and attitudinal signals captured through event instrumentation of the application suite. Instrumentation follows a shared event taxonomy so metrics are comparable across applications.

#	Metric	Definition and Signal	Collection Guidance
1	Activation rate	Percentage of new users who reach the defined 'aha moment' or complete a key first action.	Define the activation event per application; instrument as a funnel from first session.
2	Feature adoption	Percentage of users engaging with a given feature, plus time-to-first-use.	Tag feature entry points; track adoption curves per release.
3	Retention (D1/D7/D30)	Percentage of users returning over time, measured by acquisition cohort.	Cohort-based; never blend cohorts. Segment by application and user role.
4	Stickiness (DAU/MAU)	Ratio expressing how habitually the product is used.	Compute per application; interpret against expected usage rhythm (daily tool vs. periodic tool).
5	Task success rate	Percentage of users completing core workflows without failure or abandonment.	Define start/end events per core workflow; count abandonment explicitly.
6	Time to complete key tasks	Efficiency of critical flows; watch for regressions after releases.	Timestamp workflow start/end events; annotate with release version.
7	Error/friction rate	Rage clicks, dead clicks, form errors, and failed actions per session.	Session replay or behavioral analytics tooling; correlate with 4xx anomalies in Layer 2.
8	Support contact rate	Tickets per active user, per application, and per workflow. Friction that escaped instrumentation: users who hit a wall and contact support often never generate the in-product events.	Join ticketing data to the product analytics dimensions (application, workflow, release version); trend deltas against releases.
9	Session depth/breadth	Actions per session and number of distinct features touched.	Derived from the event stream; trend by cohort.
10	User satisfaction	CSAT, SUS, or NPS treated as a signal, tied to specific flows rather than the product at large. Supplemented by support-derived satisfaction (CSAT on resolved tickets, ticket sentiment), with the caveat that it samples only the unhappiest users.	In-context micro-surveys at workflow completion; avoid generic quarterly blasts. Report support-derived satisfaction separately, never blended.
11	Churn signals/dormancy	Declining usage frequency before users go fully inactive.	Define dormancy thresholds per application; alert product owners on cohort-level decline.

Where to start: Activation, retention, and task success carry the most diagnostic power with limited traffic. The remaining metrics layer in as baseline volume accumulates.

4. Layer 2: Technical Telemetry (How the Product Behaves)

Runtime reliability and performance signals. Items 1 through 5 and 7 map closely to the golden signals model (latency, traffic, errors, saturation), which provides a defensible framing for standardization across teams. Server and client error classes are tracked as separate metrics because they carry different meanings and different response models.

#	Metric	Definition and Signal	Collection Guidance
1	Availability/uptime	Percentage of time the service is operational, measured against published SLOs.	Synthetic probes plus real-traffic success rate; error budgets per service.
2	5xx server error rate	Unhandled exceptions, timeouts, crashed pods, exhausted pools, dead dependencies. These are service failures.	SLO-bound. Alert on burn-rate thresholds; baseline near zero. Pages on breach.
3	4xx client error anomaly rate	Trended, not alerted; watched as deltas correlated with deployments. Classes: 401/403 (auth and policy misconfiguration; a 403 spike after a policy change is an early warning of over-restriction), 404 (stale clients, broken links, API version mismatch), 400/422 (contract breaks and malformed payloads between services, a key integration-health signal in a distributed architecture), 429 (rate limiting from abuse or under-provisioned capacity).	Trend deltas against per-endpoint baselines; annotate deployments and policy changes. Auth-failure spikes may warrant faster response. Track server-caused 4xx (your own clients sending bad requests) as a distinct annotation; these are genuine defects and feed the Layer 3 escape rate.
4	Crash rate	Unhandled client-side crashes per session.	Crash reporting SDKs; segment by app version, OS, and device class.
5	Latency (p50/p95/p99)	Response times at percentiles, never averages. Tail latency is where users feel pain.	Histogram-based collection; report percentiles per endpoint and per cohort.
6	Frontend performance	Core Web Vitals (LCP, INP, CLS) and time-to-interactive for web surfaces.	Real-user monitoring (RUM), supplemented by lab measurement in CI.
7	Throughput	Requests and transactions per second; capacity headroom.	Per service and per endpoint; trend against provisioned capacity.
8	MTTD/MTTR	How fast incidents are detected and resolved.	Derived from incident tooling timestamps; reviewed in postmortems.
9	Saturation/resource utilization	CPU, memory, storage, and connection pools under load.	Infrastructure metrics with utilization alerts well below hard limits.
10	API health per endpoint	Latency and 5xx/4xx breakdown by service and endpoint, never aggregate alone.	Dimension every telemetry stream by service, endpoint, method, and client.
11	Client-side telemetry	JS errors, ANRs, network failures, device and browser breakdowns.	RUM and mobile telemetry SDKs; correlate with backend releases.
12	Dependency health	Third-party API latency and failures, database query performance, queue depths.	Instrument every egress call; the common blind spot is a service that looks healthy while a downstream dependency quietly degrades the experience.

Alerting doctrine: Alert on 5xx, trend on 4xx. Server errors breach SLOs and page on-call engineers. Client-error anomalies are investigated during business hours, with auth failures as the exception. Expected 4xx (for example, a user mistyping a password) is excluded from SLO math.

5. Layer 3: Programmatic Delivery (How the Organization Delivers)

Flow and quality signals for the delivery system itself. The four DORA metrics anchor this layer and live in a separate engineering-performance view, distinct from runtime telemetry. DORA's own research is explicit that these are system health indicators, not performance rankings; in a converged organization of this scale they become counterproductive quickly if read as team scorecards.

#	Metric	Definition and Signal	Collection Guidance
1	Deployment frequency	How often code ships to production. (DORA)	Derived from CI/CD pipeline events; per service and per organization baseline.
2	Lead time for changes	Commit to production; end-to-end pipeline speed. (DORA)	Commit timestamp to deployment timestamp, automated from the pipeline.
3	Change failure rate	Percentage of deployments causing incidents, rollbacks, or hotfixes. (DORA)	Link deployments to incident and rollback records; requires disciplined tagging.
4	Failed deployment recovery time	How fast a bad change is remediated. (DORA; replaced MTTR in the 2023 revision.)	Time from failure detection to remediation deployment.
5	PR cycle time	Open-to-merge duration, including time-to-first-review; the biggest hidden bottleneck in most organizations.	Source control analytics; decompose into wait states before optimizing.
6	Build/pipeline health	CI duration, flaky test rate, pipeline success rate.	Pipeline telemetry; quarantine and track flaky tests explicitly.
7	Test coverage & escape rate	Coverage as a floor; defects escaping to production as the real signal. Escapes arrive through three channels: monitoring, server-caused 4xx identified in Layer 2, and support tickets confirmed as defects.	Coverage from CI; escapes from defect tracking. Ticket-to-defect linkage requires support triage to tag tickets that trace to a shipped defect.
8	Code churn/rework	Percentage of code rewritten shortly after merge; a proxy for unclear requirements or rushed work.	Source control analytics over a defined rework window (e.g., 21 days).
9	Work in progress & throughput	Items in flight versus completed per sprint; flow efficiency.	Work management tooling; visualize as cumulative flow.
10	Work ratio	Distribution of effort across work types: new capability, debt remediation, defect fix, and keep-the-lights-on. A shrinking new-capability share is the early warning that debt and operational load are consuming the organization.	Work-item classification against a small fixed type taxonomy; trend the mix per organization, do not target a universal split.

Convergence caution: Baseline each legacy organization separately before setting any org-wide target. Starting points will differ widely, and a single early target punishes the teams that inherited the worst pipelines while teaching everyone to game the numbers.

6. Cross-Layer Linkages

The layers are only a strategy, rather than three disconnected dashboards, if they can be joined. The following linkages are first-class analysis paths and drive the shared-dimension requirement in Section 9.

Signal (Source Layer)	Connects To (Target Layer)	Question It Answers
p95+ latency and error rate, segmented by cohort (L2)	Friction rate, task success, churn signals (L1)	Is a slow or failing endpoint actually driving the friction and churn we observe?
4xx anomaly deltas after deployment (L2)	Change failure rate, escape rate (L3)	Which releases break client contracts, and is the failure rate improving?
Server-caused 4xx annotations (L2)	Escape rate (L3)	What defects are hiding in the client-error bucket?
Crash rate and client telemetry (L2)	Retention and satisfaction (L1)	Are stability problems on specific devices or versions costing us users?
Lead time and deployment frequency (L3)	Time-to-first-use of new features (L1)	Is delivery speed translating into user value, or just into deploys?
Support contact rate deltas after deployment (L1)	4xx anomalies, crash rate (L2); change failure rate (L3)	Did a release create friction that only surfaced through the human channel?
Support-detected vs. monitoring-detected incidents (L1 vs. L2)	MTTD (L2)	Do customers report incidents before monitoring does? This ratio is one of the most honest measures of telemetry maturity.

7. DORA Alignment

DORA (DevOps Research and Assessment) defines five metrics: four software delivery metrics established in the original research, plus reliability, added in 2021 to measure operational performance. This strategy adopts all five, but deliberately does not place them in a single list. The four delivery metrics measure how the organization ships change and anchor Layer 3. The fifth measures how the product behaves in production, which is precisely what Layer 2 exists to capture; rather than appearing as one more row, it is decomposed into its constituent signals and summarized as SLO attainment.

DORA Metric	Measures	Location in This Strategy
1. Deployment frequency	How often code ships to production (velocity).	Layer 3, item 1.
2. Lead time for changes	Commit to production; end-to-end pipeline speed (velocity).	Layer 3, item 2.
3. Change failure rate	Percentage of deployments causing incidents, rollbacks, or hotfixes (stability).	Layer 3, item 3.

DORA Metric	Measures	Location in This Strategy
4. Failed deployment recovery time	How fast a bad change is remediated (stability). Replaced MTTR in the 2023 revision.	Layer 3, item 4.
5. Reliability	Operational performance: the ability to meet or exceed reliability targets, spanning availability, latency, performance, and scalability. Added in 2021.	Layer 2 in its entirety, summarized as SLO attainment. Availability (item 1), 5xx rate (item 2), latency percentiles (item 5), and saturation (item 9) are its constituent signals; MTTD/MTTR (item 8) and alerting doctrine cover the operational-practice dimension.

Why the split matters: DORA's own research found that teams at every level of delivery performance see better outcomes when they also prioritize operational performance. Keeping delivery metrics in Layer 3 and reliability in Layer 2, joined through the cross-layer linkages, makes that coupling explicit instead of flattening two distinct concerns into one list.

8. Customer Support Signals

Support metrics split into two categories, and the distinction determines where they live. Signals about the product flow into the existing layers, because a support ticket is user friction that escaped instrumentation. Metrics about the support function itself form a companion operational view, kept outside the three layers for the same reason DORA lives in its own engineering-performance view: a different system under measurement, a different owner, and distinct failure modes if conflated.

Product-facing support signals (folded into the layers)

Support contact rate (Layer 1, item 8). Tickets per active user, application, and workflow. Sits beside error/friction rate and catches what behavioral analytics miss, since users who hit a wall and contact support never generate the in-product events.

Ticket-confirmed defects (Layer 3, escape rate). A ticket that traces to a shipped defect is an escaped defect, joining monitoring-detected escapes and server-caused 4xx as the third detection channel.

Release-correlated ticket spikes (annotation discipline). A step change in contact rate after a deployment is the same class of signal as a 4xx anomaly delta, arriving through a human channel with more latency. Support dashboards carry the same deployment annotations.

Support-derived satisfaction (Layer 1, item 10). CSAT on resolved tickets and ticket sentiment supplement in-product satisfaction, reported separately because they sample only the unhappiest users.

Support operations companion view

These measure how effectively the support function delivers, owned by the support lead and reviewed on its own cadence:

Metric	Definition and Signal	Collection Guidance
First response time	Time from ticket creation to first substantive human response.	Ticketing system timestamps; percentiles, not averages, per priority tier.
Resolution time	Time from creation to confirmed resolution.	Exclude time waiting on the customer; segment by ticket category.
Backlog & aging	Open tickets by age band; growth or drawdown of the queue.	Trend weekly; aging distribution matters more than raw count.
First-contact resolution	Percentage of tickets resolved without escalation or reopening.	Count reopens within a defined window as failures.
Escalation rate	Percentage of tickets escalated to engineering; a load signal on delivery teams.	Tag escalations by receiving team; feeds the keep-the-lights-on share of the Layer 3 work ratio.
Deflection rate	Percentage of contacts resolved by self-service (documentation, in-product help) before reaching a human.	Instrument help-content usage and join to subsequent ticket creation.

The observability honesty check: Track the ratio of support-detected to monitoring-detected incidents. If customers routinely report incidents before monitoring does, the telemetry has a gap; this ratio is one of the most honest measures of observability maturity available to engineering leadership.

9. Collection Requirements

Shared dimensions. Every event and metric carries release version, service, endpoint (where applicable), and an anonymized user-cohort identifier. Cross-layer joins depend on this and it cannot be retrofitted cheaply.

A common event taxonomy. Product events follow one naming convention across the application suite so Layer 1 metrics are comparable between applications from day one.

Deployment and policy-change annotations. Every dashboard that trends deltas overlays deployment markers and authorization policy changes, because step changes correlated with a change event are the primary actionable signal.

Percentiles over averages. Latency and duration metrics are collected as histograms and reported as percentiles.

SLO discipline. Availability, 5xx rate, and latency carry published SLOs with error budgets. Nothing else pages.

Privacy and least data. Behavioral analytics use anonymized or pseudonymized identifiers; collect the minimum needed to answer the defined questions.

10. Rollout Sequence

Phase 1 (Quarters 1-2): Instrument the floor. Stand up Layer 2 golden signals (availability, 5xx, latency, saturation) with SLOs on critical services, plus Layer 1 activation, retention, and task success on flagship applications. Automate the four DORA metrics from pipeline data and capture per-organization baselines. No targets yet.

Phase 2 (Quarters 2-3): Segment and correlate. Add per-endpoint breakdowns, 4xx anomaly classes with deployment annotations, crash and client telemetry, the remaining Layer 1 behavioral metrics, and the support-signal joins (contact rate, ticket-to-defect linkage, deployment annotations on support dashboards). Establish the cross-layer joins in Section 6.

Phase 3 (Quarter 4 onward): Mature the delivery view. Add PR cycle time, escape rate, churn/rework, and the work-ratio view. Set improvement goals per organization against their own baselines, reviewed quarterly. Only then consider org-wide reference ranges.

Summary: product analytics tell us what users do, technical telemetry tells us how the product behaves, and programmatic metrics tell us how the organization delivers. Collected with shared dimensions and read as trends against per-organization baselines, the three layers form a single defensible measurement architecture rather than three competing dashboards.